

CSE 5525 Final Project Midpoint

Austin Bian
bian.154@osu.edu

Brayden Chawla
chawla.114@osu.edu

Nicholas McCracken
mccracken.161@osu.edu

1 Introduction

Modern LLM chat assistants are typically post-trained through multi-stage pipelines which combine supervised fine-tuning (SFT) with human preference-based alignment. In this project, we train a provided Llama-3.2-1B base model using the Tulu3 dataset and the Tinker training infrastructure to reproduce a simplified version of this post-training pipeline. The objective is to build a correct and reproducible training pipeline that utilizes SFT and Direct Preference Optimization (DPO) to reshape the model's behavior to human preferences.

2 Methodology

We will begin by performing supervised fine-tuning on the Tulu 3 SFT Olmo 2 Mixture - 0225 dataset using parameter-efficient fine-tuning to ensure computational efficiency. This stage converts the pre-trained base model into an instruction-following assistant capable of producing structured and coherent responses. We use LoRA with rank 32 for parameter-efficient fine-tuning, meaning the base model weights are frozen and only the low-rank adapter matrices are trained. After training, the LoRA adapter weights are merged back into the base model to produce a standalone fine-tuned checkpoint.

Training was done using Tinker, an api designed to assist with training and finetuning machine learning models. Most of the logic for training the model including resource-intensive procedures such as the forward pass, optimizer step, model checkpoint, and data sampling, were handled by Tinker, while the construction of complete model, evaluation, and inference was handled locally. The resulting model will be evaluated on the fixed General, Safety, Math, and Coding evaluation splits using the provided evaluation framework, without training on any evaluation prompts.

Starting from our best SFT checkpoint, we will then apply Direct Preference Optimization using the required preference dataset. We will compare three models: the base model, the SFT model, and the SFT + DPO model. This controlled comparison will allow us to isolate the behavioral impact of alignment and measure changes in instruction-following accuracy, safety compliance, and robustness.

3 Prior Work / Related Work

Modern post-training pipelines rely heavily on supervised tuning and preference-based alignment. InstructGPT, created by Ouyang et al., established the industry standard when they found that their fine-tuned model performed better than GPT-3 with over 100x fewer parameters (Ouyang et al.). The drawback with this model, however, is that it uses reinforcement learning from human feedback, which is highly unstable and resource intensive. This issue is addressed by utilizing direct preference optimization (Rafailov et al.). Direct preference optimization was found to be more stable, lightweight, and reproducible when creating a training pipeline (Rafailov et al.). What Rafailov et al. fail to address is what are the unintended tradeoffs created by this methodology. Our project will explicitly address this gap by documenting and quantifying these behavioral shifts when comparing the DPO-aligned model directly against a baseline SFT model.

Anderson et Al. has shown results on the aforementioned Llama-3.2-1B model with supervised fine-tuning, and published their results in the Olmo 2 paper. Their results show strong performance by Llama-3.2-1B on various different benchmarks, including GSM8K, a dataset we are evaluating on. They propose different hyperparameter configurations related to our work, such as the absence of weight decay and a learning rate between 0.0003

and 0.0006.

4 Supervised Fine-Tuning (SFT)

4.1 Hyperparameter Exploration

We trained several SFT model variants to understand how dataset size, sequence length, and learning rate affect downstream performance. By utilizing different configurations in hyperparameters we could test different approaches to increasing performance for the evaluation benchmarks. All runs share a common base configuration (LoRA rank 32, batch size 128, single epoch) and differ only in the parameters shown in the table. We also used a test run with a configuration of 15 steps for testing the functionality of the pipeline while keeping training costs low. The other configurations explore trade-offs between training on more data versus less or quality differences from choices of learning rate and max length.

Model	Steps	Batch Size	Length	LR	LoRA
Test	15	128	16384	4.9e-4	32
Partial	1500	128	4096	4.9e-4	32
Full	6758	128	4096	4.9e-4	32

Table 1: Model configs.

The selected hyperparameters were chosen based on prior work and practical constraints as well. A relatively high learning rate (4.9e-4) was used to encourage faster convergence given the looming budget constraints of the project. The LoRA rank of 32 was best at balancing parameter efficiency and performance of the model, and the batch size of 128 was recommended by Tinker to use within their infrastructure (7).

We varied the number of training steps and dataset size to study the tradeoff between the duration of training and the generalized performance. The test configuration with 15 steps serves as a pipeline validation run, to ensure correctness of our implementation before committing time, budget, and resources to a model that may be working unexpectedly. The partial and full configurations explored whether increasing exposure to the training data leads to improved downstream performance or introduces overfitting to the model.

4.2 Results

We evaluate all trained models across multiple benchmarks to assess their performance on different instruction following, reasoning, coding, and safety tasks; all the primary, modern use-cases for

an LLM. These benchmarks are chosen to provide a comprehensive view of the model capabilities in totality, as success in one domain does not necessarily transfer to all domains.

Table 2 reports the primary evaluation metrics for each model configuration. The partial dataset model achieves the best overall performance across all benchmarks, which suggests that moderate training may provide better balance and avoids overfitting. This could also potentially suggest errors with our model digesting large contexts, which will be further experimented on after this checkpoint.

Model	GSM8K	MBPP	IFE	HBench	XStest
Partial	0.084	0.218	0.373	0.834	0.796
Full	0.064	0.194	XXXX	XXXX	XXXX

Table 2: Primary benchmark scores for the Partial model. Scores are rounded to three decimals.

To better understand the capabilities of the model, we can analyze performance across the fine-grained categories provided by IFEval. As shown in Table 3, the model achieves relatively strong performance on general instruction tasks, and certain form fitting task. However, the model struggles significantly on more precise formatting constraints. Performance such as JSON and bullet list formatting generation is near zero. By our current SFT model objectives, this rigid formatting is not strongly enforced. The objectives prove strong for general tasks, but weaker for specific ones.

Category	Strict	Loose
Prompt-level	0.222	0.244
Instruction-level	0.344	0.373
Length Constraints	0.577	0.596
Keywords	0.405	0.429
Formatting (Title)	0.703	0.703
JSON Format	0.0	0.0
Bullet Lists	0.0	0.0
Overall	–	0.373

Table 3: IFEval performance broken down by instruction category.

4.3 Analysis

As can be seen, the results section is not complete, as the full model was not evaluated on IFEVAL, Harmbench, or XStest. This was due to the model’s performance on GSM8K and MBPP, as the model performed even more poorly than the partial model. The performance issue of the full

model was likely due to overfitting or overshooting the model weights. There was a lack of a validation set, which could have allowed the model to memorize the dataset rather than generalizing. Additionally, there was a high learning rate of 0.00049, which, when paired over 6000 steps and linear decay, causes the model to have a higher learning rate for a longer period of time, allowing overshooting of weights.

The partial model performed well on XSTest and Harmbench, showing that the model is consistent and responds well to attacks. However, the model does not perform that well on any other metric, meaning that, when it does respond, its responses may not be logical.

Otherwise, any model that was trained has an unexpectedly low evaluation score on GSM8K at 0.08, while the expected result would have been an accuracy above 0.2. This discrepancy was likely due to zero-shot prompting during the evaluation of the model, as zero-shot prompting provided no context for the model to reason off of. Prior work, as shown in Anderson et al. (2025), demonstrated that the GSM8K dataset should be evaluated using eight-shot prompting, which would have likely caused a large increase in accuracy.

Further insight can be gained from the IFEval category breakdown. The model performs relatively well on general instruction following tasks and loosely constrained formatting requirements, such as length constraints and title generation. However, performance drops significantly on tasks requiring strict formatting, such as JSON generation and bullet list formatting. These tasks require precise control over the structure of the model’s output, something which is not strongly enforced by the SFT objective. Overall, these results suggest that the SFT model is optimized for these general tasks. The limitations are something to consider when moving into the next stage of our pipeline, where the alignment methods like DPO could help refine some of the model behavior, and improve performance in the weaker IFEval categories.

5 Next Steps

Following this checkpoint, we will extend our pipeline by implementing Direct Preference Optimization (DPO) using the provided preference dataset. Then, we can compare the baseline, SFT model, and SFT + DPO model to isolate the effects of each alignment on model behavior. We will fur-

ther refine our evaluation setup, particularly around reasoning benchmarks like GSM8K, by incorporating few-shot prompting to obtain more accurate estimates of model performance. Additionally, we will conduct further hyperparameter tuning with more runs to maximize SFT performance prior to the alignment. Finally, we will implement our proposed extensions to gain deeper insights into how training data and alignment objects influence our model. This is further discussed in the following section.

6 Proposed Extensions

After completing the required SFT and DPO pipeline, we propose two extensions designed to deepen our understanding of fine tuning and alignment tradeoffs.

6.1 Data Filtering and Quality Analysis

For extension one, we will perform data filtering and quality analysis on the instruction and preference datasets. The dataset will be analyzed at first to understand its composition, examining properties such as response length distributions, duplicate and near-duplicate frequency, and stylistic variations. Based on this analysis, we will construct filtered subsets of data using several possible approaches: length filtering to remove outlier responses, deduplication, and quality-based post training heuristics such as filtering out high-perplexity responses under the base model. For each filter, we will form a hypothesis about its expected effect on model behavior before training. We will then train models on both the full and filtered datasets and compare their behavior across all four evaluation splits to study how data quality and composition influence both supervised fine-tuning and alignment outcomes. We will select two to three well-justified filters rather than exhaustively testing all possible combinations due to possible resource constraints.

6.2 Safety vs Helpfulness Tradeoff Curves

The second extension will quantify the tradeoff of safety versus helpfulness through testing variations in the alignment strength parameter during DPO training. For each variation, the rate of refusal on safety prompts and task performance on general benchmarks will be tracked and plotted. This will allow us to observe how increasing alignment pressure affects the delicate balance between safety and helpfulness.

All experiments will be conducted with credit budget and reproducibility in mind. By documenting configurations, hyperparameters, random seeds, and evaluation outputs, we can ensure the results will be easily replicated and compared.

6.3 Evaluating Error Results

The third extension performed will be a structured qualitative analysis of categorized failure modes across SFT and SFT+DPO models for the general, math, and coding evaluation splits. We will randomly sample a fixed percentage of generated responses and manually classify using human analysis each into error categories. We propose the error categories of incomplete responses, off-topic generation, formatting errors, and general hallucination and are subject to change if further experimentation reveals any other classes needed. By comparing the distribution of error types between SFT and DPO models, we can gain insights into which failure types are introduced or fixed during the alignment stages. We will summarize the results by plotting the error type distributions for each model as a comparison, highlighting which categories shift most between SFT and DPO.

References

- [1] Rafailov, Rafael, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model.
- [2] Ouyang, Long, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback.
- [3] Lambert, Nathan, et al. 2024. Tülu 3: Pushing Frontiers in Open Language Model Post-Training.
- [4] Groeneveld, Dirk, et al. 2024. OLMo: Accelerating the Science of Language Models.
- [5] Dubey, Abhimanyu, et al. 2024. The Llama 3 Herd of Models.
- [6] Anderson, T., et al. 2025. 2 OLMo 2 Furious.
- [7] Thinking Machines Lab, *Supervised Learning Hyperparameters*, <https://tinker-docs.thinkingmachines.ai/supervised-learning/sl-hyperparams>, 2025, accessed March 2026.